BUSINESS ANALYST - TECHNICAL ASSESSMENT

Question Set: A

Congratulations on clearing the aptitude round. This assessment has four sections, all built on a single CSV. Section 2 is a substantial CASE STUDY - read its briefing carefully. The mini-project (Section 4) is the only open-ended deliverable; everything else has specific, verifiable answers.

Suggested time: 48 hours. Read all nine footnotes before starting. Submit your work as a single ZIP containing an answer document with your final answers, visualizations, and code (see Deliverables at the end).

The Dataset

A single CSV (hotel_bookings.csv) of 12,000 booking records from a hotel-booking platform. Each row is one booking; customer and property attributes are denormalized into the row. 28 columns:

- **booking_id, customer_id, property_id** – Primary key + foreign keys
- **customer_name, customer_segment, customer_signup_date, customer_home_city, customer_loyalty_tier** – Customer attributes (denormalized into every row)
- **property_name, property_city, property_star_rating, property_type, property_total_rooms** – Property attributes (denormalized into every row)
- **booking_date, checkin_date, checkout_date** – Key dates
- **room_type, num_rooms, nights** – Room details
- **booking_channel** – Direct Website / OTA / Travel Agent / Corporate Portal
- **adr, discount_amount, coupon_code, total_amount, payment_method** – Pricing & payment. Stated formula: total_amount = adr x nights x num_rooms - discount_amount
- **booking_status** – Completed / Cancelled / No-Show
- **review_rating, review_date** – Review info (blank if no review left)

Important Footnotes - read before starting

Read these before you begin. They are not a complete list of everything in the data.

Footnote 1 - Invalid stays: Some rows have checkout_date on or before checkin_date. Treat as data errors.

Footnote 2 - Booking before account: Some rows have booking_date earlier than customer_signup_date - a temporal-integrity issue.

Footnote 3 - Zero rooms: Some rows have num_rooms = 0 (carry total_amount = 0). These are erroneous.

Footnote 4 - 'No coupon' has TWO encodings: coupon_code uses BOTH an empty value AND the literal text 'NONE' to mean no-coupon. Treat both alike.

Footnote 5 - Property names repeat across cities: A few property_name values appear in multiple cities with DIFFERENT property_ids. NEVER group on property_name - always use property_id.

Footnote 6 - Reviews on Cancelled bookings: A small number of Cancelled bookings carry a review_rating, which is impossible. Flag these.

Footnote 7 - Review scale mismatch: review_rating is on 1-5 for Individual and Group customers, but on 1-10 for Corporate customers. Without normalization, any cross-segment rating average is wrong.

Footnote 8 - customer_loyalty_tier 'None' parsing trap: The tier column has 4 valid values including the literal string 'None' (meaning 'no tier'). Default CSV readers (pandas read_csv, etc.) silently convert

'None' to a missing value, making ~46% of your tier data disappear. Use keep_default_na=False (pandas) or the equivalent.

Footnote 9 - Realized vs booked revenue: Only booking_status = 'Completed' counts as realized revenue. Cancelled and No-Show rows still carry a total_amount (the would-have-been-charged amount); summing all rows overstates revenue by ~28%.

Section 1 - Data Forensics (quick)

Fast factual checks. DQ1-DQ3 each have a specific answer. DQ4 is a short discovery task.

1. DQ1. How many bookings have invalid stays (checkout_date <= checkin_date)? (integer)
2. DQ2. Compute review_rating range (min, max) and mean SEPARATELY for each customer_segment. What does this reveal? (1-line implication)
3. DQ3. After applying the relevant data-quality footnotes, what is realized revenue for property_type = Luxury? State exactly which cleaning rules you applied. (Rs)

DQ4 (discovery - applies to all sets). At least one data-integrity problem exists in this dataset that is NOT described in any footnote. Find one such undocumented issue, describe it, quantify how many rows are affected, and propose a rule for handling it. The nine footnotes are not an exhaustive list of everything wrong with this data.

Section 2 - Case Study: The Cancellation Crisis

Briefing:

The Operations head has flagged that the platform's overall booking cancellation rate is around 19-20% - well above the industry benchmark of 12%. The board has set a target to reduce platform-wide cancellation by 5 percentage points within two quarters. They expect you to identify where the problem is concentrated, diagnose the underlying cause from the data, and recommend a targeted intervention with a quantified expected impact. A vague 'we should improve our policies' will not pass. Numbers and what-if calculations will.

What we expect:

You must include at least TWO labelled visualizations: one for the landscape view (CS1) and one for the diagnosis (CS3 or CS4). Every recommendation must come with explicit arithmetic - if a number isn't shown, it doesn't count.

CS1 - Cancellation Landscape (visualization required).

Build a clear visual of cancellation rate broken down by AT LEAST TWO dimensions at once (for example property_city x calendar month as a heatmap, OR booking_channel x lead-time bucket, where lead time = days between booking_date and checkin_date). State which dimension(s) drive the most variance in cancellation rate - i.e. which slice most strongly separates high-cancel from low-cancel bookings.

CS2 - Rate vs Volume.

Rank slices two ways: (i) the TOP 3 slices by cancellation RATE, and (ii) the TOP 3 slices by absolute cancellation COUNT. Are the high-rate slices the same as the high-count slices? In 2-3 lines, explain why this distinction matters for the board's 5-percentage-point target.

CS3 - Root Cause: Hypothesis Testing.

For your single worst-rate slice, test AT LEAST TWO competing explanations against the data, side by side: (i) a lead-time effect (compare the lead-time distribution of the slice vs the rest of the platform); (ii) a channel-mix effect (compare the channel mix of the slice vs the rest); (iii) a genuine city-and-season combination (compare the slice's rate against the same city in OTHER months AND a different city in the SAME months). For each hypothesis, show the comparison numbers. State which the data best supports and how you ruled the others out. Apply the relevant data-quality footnotes before drawing any conclusion.

CS4 - What-if Calculation.

Compute two what-if scenarios (show the arithmetic): (i) if your worst-rate slice were brought down to the PLATFORM-WIDE average rate, how many cancellations are prevented, and what is the reduction in percentage points of the platform-wide rate? (ii) the same exercise for a long-lead intervention (assume all bookings with lead time over 30 days are brought down to the cancellation rate of bookings with lead time 30 days or fewer). Which intervention has the larger absolute impact, and why?

CS5 - Targeted Recommendation.

Propose ONE specific, data-grounded intervention. Specify (a) WHO is targeted (city, channel, segment, lead-time - be precise), (b) WHAT the policy is (e.g. non-refundable deposit, partial-refund tier, weather-indexed insurance), and (c) the QUANTIFIED expected impact in percentage points of the platform-wide cancel rate, stated as a range. Name one risk / side-effect.

CS6 - Is the board's target achievable? (honesty check).

Using your what-if numbers from CS4, state plainly whether the board's 5-percentage-point target is achievable from interventions the data supports. If yes, show the combination that gets there. If not, state the realistic ceiling and what additional data or levers (outside this dataset) would be needed. An honest, quantified 'no, the data does not support 5 points' will score HIGHER than an optimistic answer whose math does not add up.

Section 3 - SQL Challenge

Tests SQL fluency AND schema understanding. The dataset is denormalized; your first task is to design a normalized schema. The queries are written against the schema YOU design.

Schema Design Task (worth as much as the queries):

Design a normalized schema by writing CREATE TABLE statements for at least three tables (customers, properties, bookings, optionally reviews). For every column specify:

- SQL data type (INTEGER, VARCHAR(n), DECIMAL(p,s), DATE, etc.) - think precision for money and length for free text;
- primary key, foreign key, or neither - plus NOT NULL where appropriate;
- TWO non-trivial CHECK or UNIQUE constraints, each addressing one footnote (e.g. CHECK(checkout_date > checkin_date) for Footnote 1; UNIQUE(property_name, property_city) for Footnote 5);
- ONE column to index for speeding up your queries below, with a 1-2 line justification.

SQL Queries (against your normalized schema, or the flat CSV loaded as one table - state which):

A-Q1.

For each property_city, return the property_id, property_name, and revenue of the property with the HIGHEST realized revenue. Use RANK() OVER (PARTITION BY property_city ORDER BY revenue DESC). Group correctly given Footnote 5 (property names repeat across cities).

A-Q2.

Using LAG() over each customer's Completed bookings ordered by checkin_date, compute the gap in days between consecutive bookings per customer. Return the COUNT of customers whose AVERAGE gap is under 30 days.

Section 4 - Mini-Project (build something + integrate a free API)

The only open-ended part. Build a small working tool or notebook that enriches the dataset using a live, free, no-key public API and produces one concrete, quantified insight.

Set A - Weather-Cancellation Analyzer

API: Open-Meteo Historical Weather API - free, no key. Endpoint: https://archive-api.open-meteo.com/v1/archive (parameters latitude, longitude, start_date, end_date, daily=precipitation_sum,temperature_2m_max).

What to build: For a sensible sample of bookings (e.g. all Goa and Manali check-in dates in 2024), fetch the actual weather on the check-in date and test whether heavy-rain or extreme-temperature days are associated with higher cancellation or no-show rates. Map each city to coordinates yourself and batch your calls sensibly (do not hit the API once per row - group by city + date).

Every mini-project must include ALL of the following:

1. A real call to the assigned free API for your set (Set A: Open-Meteo; Set B: Nager.Date; Set C: Frankfurter), with error handling so a timeout or missing data does not crash the run. If you use a different API than the one assigned to your set, this section scores zero.
2. A correct merge of the API data back onto the bookings (watch join keys, dates, and city mappings).
3. ONE non-obvious, quantified insight relevant to your set that the dataset ALONE could not produce - a number or chart, stated in 3-4 sentences.
4. A 6-line README: what it does, how to run it, one design decision, one limitation.

Deliverables

Submit a single ZIP containing:

1. An answer document (PDF, Word, or notebook) containing your final answers for Section 1 and Section 2, your visualizations, and your schema and SQL queries.
2. A code/ folder with your analysis notebook/script and SQL files.
3. A project/ folder with your working mini-project code, its README, and the insight it produced.